

# Python Code Challenge



Algorithmisches Denken & Objektorientierte Programmierung

Vorlesungsskript für KI-Studierende

*Kannst du lesen, was der Code macht?*

## **So funktioniert's:**

1. Lies den Code aufmerksam durch.
2. Spiele den Code im Kopf (oder auf Papier) Zeile für Zeile durch.
3. Wähle die richtige Antwort aus A, B, C oder D.
4. Die Lösungen findest du jeweils nach dem Aufgabenteil.

# Inhaltsverzeichnis

---

|                                                            |    |
|------------------------------------------------------------|----|
| Inhaltsverzeichnis .....                                   | 2  |
| Teil 1: Algorithmisches Denken – Grundlagen .....          | 3  |
| Aufgaben .....                                             | 3  |
| Lösungen zu Teil 1 .....                                   | 6  |
| Teil 2: Code-Interpretation – Funktionale Muster .....     | 8  |
| 2.1 Die Schatzsuche – Maximum in einer Liste .....         | 8  |
| 2.2 Der Türsteher – Gerade Zahlen filtern .....            | 8  |
| 2.3 Der Vokal-Zähler – Strings analysieren .....           | 9  |
| 2.4 Der Spiegel – Palindrom-Prüfung .....                  | 9  |
| Teil 3: Weiterführende Algorithmen-Challenges .....        | 10 |
| Lösungen zu Teil 3 .....                                   | 12 |
| Teil 4: Objektorientierung I – Klassen und Instanzen ..... | 14 |
| Aufgaben .....                                             | 14 |
| Teil 5: Code-Interpretation – OOP-Bausteine .....          | 17 |
| 5.1 Die Batterie – Konstruktor und Zustand .....           | 17 |
| 5.2 Der Rucksack – Objekte und Listen .....                | 17 |
| 5.3 Der Vampir – Interaktion zwischen Objekten .....       | 18 |
| 5.4 Das Echo – Dunder-Methoden .....                       | 18 |
| Teil 6: OOP II – Beziehungen zwischen Klassen .....        | 19 |
| 6.1 Das Upgrade – Vererbung (is-a) .....                   | 19 |
| 6.2 Der flüchtige Kontakt – Assoziation .....              | 19 |
| 6.3 Der Club – Aggregation (has-a, lose) .....             | 20 |
| 6.4 Der Motorblock – Komposition (has-a, streng) .....     | 20 |
| Anhang: Klassische Algorithmen im PyTorch-Kontext .....    | 21 |

## Teil 1: Algorithmisches Denken – Grundlagen

---

In diesem ersten Teil trainierst du das schrittweise Nachvollziehen von Python-Code. Die Challenges decken grundlegende Konzepte ab: Schleifen, Strings, Listen, Dictionaries und klassische Algorithmen. Lies den Code sorgfältig und überlege, was Zeile für Zeile passiert.

### Aufgaben

#### Challenge 1.1

##### » HAL → ??? «

Kategorie: Zeichenkodierung (ASCII)

Was gibt dieser Code aus?

```
text = "HAL"
result = ""
for char in text:
    result += chr(ord(char) + 1)
print(result)
```

Antwortmöglichkeiten:

- A) "IBM"
- B) "HAM"
- C) "GZK"
- D) "HAL1"

#### Challenge 1.2

##### » Die Doppelgänger-Liste «

Kategorie: Referenzsemantik

Was gibt dieser Code aus?

```
a = [1, 2, 3]
b = a
b.append(4)
print(len(a))
```

Antwortmöglichkeiten:

- A) 3
- B) 4
- C) Fehlermeldung
- D) [1, 2, 3]

## Challenge 1.3

### » Der Wortzähler «

Kategorie: Dictionaries

Was gibt dieser Code aus?

```
text = "die katze und die maus und die maus"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts["maus"])
```

Antwortmöglichkeiten:

- A) 1
- B) 2
- C) 3
- D) Fehlermeldung (KeyError)

## Challenge 1.4

### » Die Collatz-Vermutung «

Kategorie: Bedingte Schleifen

Was gibt dieser Code aus?

```
n = 6
steps = 0
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = n * 3 + 1
    steps += 1
print(steps)
```

Antwortmöglichkeiten:

- A) 6
- B) 8
- C) 10
- D) Endlosschleife – das Programm hält nie an

## Challenge 1.5

### » Euklids Algorithmus «

Kategorie: Mathematische Algorithmen

Was gibt dieser Code aus?

```
a = 48
b = 18
while b != 0:
    a, b = b, a % b
print(a)
```

Antwortmöglichkeiten:

- A) 2
- B) 6
- C) 18
- D) 0

## Challenge 1.6

### » Die vergessliche Funktion «

Kategorie: Mutable Default Arguments

Was gibt dieser Code aus?

```
def sammle(item, liste=[]):
    liste.append(item)
    return liste

print(sammle("Apfel"))
print(sammle("Birne"))
```

Antwortmöglichkeiten:

- A) ["Apfel"] und ["Birne"]
- B) ["Apfel"] und ["Apfel", "Birne"]
- C) Fehlermeldung
- D) [] und ["Apfel"]

## Lösungen zu Teil 1

### Challenge 1.1: HAL → ???

**Richtige Antwort: A) "IBM"**

#### Schritt für Schritt:

```
H → ord(72) + 1 = 73 → chr(73) = I
A → ord(65) + 1 = 66 → chr(66) = B
L → ord(76) + 1 = 77 → chr(77) = M
```

**Hinweis:** Jeder Buchstabe wird um 1 im ASCII-Alphabet verschoben: H→I, A→B, L→M. Fun Fact: In Kubricks «2001: Odyssee im Weltraum» heißt der Computer HAL 9000 – jeder Buchstabe liegt genau eins vor IBM.

### Challenge 1.2: Die Doppelgänger-Liste

**Richtige Antwort: B) 4**

#### Schritt für Schritt:

```
a = [1, 2, 3]      # a zeigt auf ein Listen-Objekt
b = a              # b zeigt auf DASSELBE Objekt
b.append(4)         # Liste wird [1, 2, 3, 4]
len(a)              # 4 (a zeigt auf dieselbe Liste!)
```

**Hinweis:** `b = a` erzeugt keine Kopie! Beide Variablen referenzieren dasselbe Objekt im Speicher. Änderungen über `b` wirken sich auch auf `a` aus. Für eine echte Kopie: `b = a.copy()` oder `b = a[:]`.

### Challenge 1.3: Der Wortzähler

**Richtige Antwort: B) 2**

#### Schritt für Schritt:

```
Schleife durchläuft: "die", "katze", "und", "die", "maus", "und", "die", "maus"
counts = {"die": 3, "katze": 1, "und": 2, "maus": 2}
counts["maus"] = 2
```

**Hinweis:** `text.split()` zerlegt den String in Einzelwörter. Das Dictionary `counts` speichert für jedes Wort seine Häufigkeit. Die Methode `.get(word, 0)` gibt 0 zurück, falls das Wort noch nicht enthalten ist – so wird kein `KeyError` ausgelöst.

## Challenge 1.4: Die Collatz-Vermutung

**Richtige Antwort: B) 8**

### Schritt für Schritt:

```
n=6 (gerade) → 3 → Schritt 1
n=3 (ungerade) → 10 → Schritt 2
n=10 (gerade) → 5 → Schritt 3
n=5 (ungerade) → 16 → Schritt 4
n=16 → 8 → Schritt 5
n=8 → 4 → Schritt 6
n=4 → 2 → Schritt 7
n=2 → 1 → Schritt 8 ✓ fertig!
```

**Hinweis:** Die Collatz-Folge: Ist n gerade, wird halbiert; ist n ungerade, wird mit 3 multipliziert und 1 addiert. Die berühmte Vermutung besagt, dass jede Startzahl irgendwann bei 1 ankommt – bewiesen ist das bis heute nicht!

## Challenge 1.5: Euklids Algorithmus

**Richtige Antwort: B) 6**

### Schritt für Schritt:

```
a=48, b=18 → a, b = 18, 48%18 = 18, 12
a=18, b=12 → a, b = 12, 18%12 = 12, 6
a=12, b=6 → a, b = 6, 12%6 = 6, 0
b=0 → Schleife endet → print(6)
```

**Hinweis:** Dieser über 2000 Jahre alte Algorithmus berechnet den größten gemeinsamen Teiler (ggT). Die Zeile `a, b = b, a % b` ist eine gleichzeitige Zuweisung – eine elegante Python-Spezialität! Der ggT von 48 und 18 ist 6.

## Challenge 1.6: Die vergessliche Funktion

**Richtige Antwort: B) ["Apfel"] und ["Apfel", "Birne"]**

### Schritt für Schritt:

```
Erster Aufruf: liste (Default) ist []
               → append("Apfel") → liste = ["Apfel"]
Zweiter Aufruf: liste ist IMMER NOCH dasselbe Objekt
               → append("Birne") → liste = ["Apfel", "Birne"]
```

**Hinweis:** Die berüchtigtste Python-Falle! Der Default-Wert `liste=[]` wird nur einmal erzeugt – bei der Definition der Funktion, nicht bei jedem Aufruf. Beide Aufrufe teilen sich dasselbe Listenobjekt. Korrekter Ansatz: `def sammle(item, liste=None): if liste is None: liste = []`

## Teil 2: Code-Interpretation – Funktionale Muster

---

In diesem Teil interpretierst du kleine Funktionen, deren Zweck sich erst beim genauen Hinsehen offenbart. Hier geht es weniger um Multiple Choice als um das Verstehen funktionaler Bausteine.

### 2.1 Die Schatzsuche – Maximum in einer Liste

Dieses Snippet versteckt die klassische Suche nach dem größten Wert hinter einem sprechenden Variablennamen. Es zeigt, wie Variablen im Hintergrund aktualisiert werden.

```
def mystery_function_A(kisten):  
    wert = kisten[0]  
    for k in kisten:  
        if k > wert:  
            wert = k  
    return wert
```

**Was es macht:** Die Funktion findet die größte Zahl in einer Liste.

**Lernziel:** Schleifen (for), Vergleichsoperatoren und das bedingte Überschreiben von Variablenzuständen.

### 2.2 Der Türsteher – Gerade Zahlen filtern

Hier kommt, ähnlich wie bei FizzBuzz, der Modulo-Operator ins Spiel. Anfänger tun sich damit oft schwer – daher ist dies eine ideale Übung.

```
def mystery_function_B(gaeste):  
    drinnen = []  
    for g in gaeste:  
        if g % 2 == 0:  
            drinnen.append(g)  
    return drinnen
```

**Was es macht:** Die Funktion filtert eine Liste und behält nur die geraden Zahlen.

**Lernziel:** Den Modulo-Operator (Restwertdivision) und das schrittweise Befüllen einer neuen, anfangs leeren Liste.



## 2.3 Der Vokal-Zähler – Strings analysieren

Hier wird nicht gerechnet, sondern Text analysiert. Das zeigt, dass Code nicht nur für Mathematik da ist.

```
def mystery_function_C(wort):
    zaehler = 0
    for buchstabe in wort:
        if buchstabe.lower() in "aeiou":
            zaehler = zaehler + 1
    return zaehler
```

**Was es macht:** Die Funktion zählt, wie viele Vokale in einem Wort oder Text enthalten sind.

**Lernziel:** Iteration durch Strings, Methodenaufrufe (.lower()) und das schrittweise Hochzählen einer Variablen.

**Hinweis:** Achtung: Die Funktion zählt nur die fünf englischen Vokale (a, e, i, o, u). Deutsche Umlaute (ä, ö, ü) werden nicht erkannt. Für eine vollständige Lösung müsste der Vergleichsstring um diese Zeichen ergänzt werden: "aeiouäöü".

## 2.4 Der Spiegel – Palindrom-Prüfung

Eine etwas härtere Nuss für das logische Denken, da man von zwei Seiten gleichzeitig auf ein Wort schaut.

```
def mystery_function_D(wort):
    laenge = len(wort)
    for i in range(laenge // 2):
        if wort[i] != wort[laenge - 1 - i]:
            return False
    return True
```

**Was es macht:** Die Funktion prüft, ob ein Wort ein Palindrom ist (z. B. "ANNA" oder "LAGERREGAL").

**Lernziel:** Index-Zugriffe (wort[i]), Booleans (True/False) und das vorzeitige Beenden einer Funktion mit return.

## Teil 3: Weiterführende Algorithmen-Challenges

---

Diese Challenges vertiefen das algorithmische Denken anhand praxisnaher Muster: Extremwertsuche, Partitionierung, zyklische Verschiebung und endliche Automaten.

### Challenge 3.1

#### » Der Highlander «

Kategorie: Extremwertsuche (Minimum)

Was gibt dieser Code aus?

```
werte = [15, 8, 23, 4, 42]
boss = werte[0]
for w in werte[1:]:
    if w < boss:
        boss = w
print(boss)
```

Antwortmöglichkeiten:

- A) 15
- B) 42
- C) 4
- D) 8

### Challenge 3.2

#### » Die Ausmusterung «

Kategorie: Partitionierung

Was gibt dieser Code aus?

```
daten = [12, 45, 7, 23, 56, 19]
limit = 20
links = []
rechts = []
for x in daten:
    if x <= limit:
        links.append(x)
    else:
        rechts.append(x)
print(len(links), len(rechts))
```

Antwortmöglichkeiten:

- A) 3 3
- B) 2 4
- C) 4 2
- D) 6 0

## Challenge 3.3

### » Das Hütchenspiel «

Kategorie: Permutation und Rotation

Was gibt dieser Code aus?

```
stapel = [10, 20, 30, 40]
for i in range(len(stapel) - 1):
    ziel = i + 1
    stapel[i], stapel[ziel] = stapel[ziel], stapel[i]
print(stapel)
```

Antwortmöglichkeiten:

- A) [40, 30, 20, 10]
- B) [20, 10, 40, 30]
- C) [20, 30, 40, 10]
- D) [10, 20, 30, 40]

## Challenge 3.4

### » Der HTML-Stripper «

Kategorie: Endliche Automaten

Was gibt dieser Code aus?

```
text = "A<b>C</b>D"
ergebnis = ""
zustand = 0 # 0 = Text aufnehmen, 1 = Tag ignorieren
for zeichen in text:
    if zeichen == '<':
        zustand = 1
    elif zeichen == '>':
        zustand = 0
    else:
        if zustand == 0:
            ergebnis += zeichen
print(ergebnis)
```

Antwortmöglichkeiten:

- A) "ACD"
- B) "A<b>C</b>D"
- C) "ABCD"
- D) "A C D"

## Lösungen zu Teil 3

### Challenge 3.1: Der Highlander

**Richtige Antwort: C) 4**

```
Start: boss = 15
8 < 15? Ja → boss = 8
23 < 8? Nein
4 < 8? Ja → boss = 4
42 < 4? Nein
Ergebnis: 4
```

**Hinweis:** Standard-Algorithmus zur linearen Minimumsuche mit einer Zeitkomplexität von  $O(n)$ . Die Variable boss speichert kontinuierlich den bisher kleinsten gefundenen Wert.

### Challenge 3.2: Die Ausmusterung

**Richtige Antwort: A) 3 3**

```
limit = 20
12 ≤ 20 → links      | 45 > 20 → rechts
7  ≤ 20 → links      | 23 > 20 → rechts
19 ≤ 20 → links      | 56 > 20 → rechts
links = [12, 7, 19] (Länge 3)
rechts = [45, 23, 56] (Länge 3)
```

**Hinweis:** Die anschaulichste Form der Partitionierung: Eine Datenmenge wird anhand einer festen Grenze in zwei Gruppen aufgeteilt – genau wie bei der Aufteilung in Trainings- und Testdaten beim Machine Learning.

### Challenge 3.3: Das Hütchenspiel

**Richtige Antwort: C) [20, 30, 40, 10]**

```
i=0: Tausche Index 0 und 1 → [20, 10, 30, 40]
i=1: Tausche Index 1 und 2 → [20, 30, 10, 40]
i=2: Tausche Index 2 und 3 → [20, 30, 40, 10]
```

**Hinweis:** Dieses Snippet zeigt eine zyklische Linksverschiebung: Das erste Element wird schrittweise bis ganz nach hinten durchgereicht. Der echte, unverzerzte Mischalgorithmus ist der Fisher-Yates-Shuffle (Knuth-Shuffle), der in jedem Schritt den Pool der Tauschpartner verkleinert – so arbeitet auch PyTorch im Hintergrund.

### Challenge 3.4: Der HTML-Stripper

**Richtige Antwort: A) "ACD"**

```
"A" aufgenommen      (ergebnis = "A")
"<" → zustand = 1    (Tag-Modus)
"b" ignoriert
">" → zustand = 0    (Text-Modus)
"C" aufgenommen      (ergebnis = "AC")
"<" → zustand = 1
"/" ignoriert
"b" ignoriert
">" → zustand = 0
"D" aufgenommen      (ergebnis = "ACD")
```

**Hinweis:** Dies ist die Minimalversion eines deterministischen endlichen Automaten (DFA) mit zwei Zuständen. Er bildet die Grundlage für das Verständnis von regulären Ausdrücken (Regex) und Compilern.

## Teil 4: Objektorientierung I – Klassen und Instanzen

Hier beginnt die Reise in die objektorientierte Programmierung. Die Challenges führen schrittweise ein: Klassen, Konstruktoren, self, Instanz- vs. Klassenvariablen, Vererbung und Polymorphie.

### Aufgaben

#### Challenge 4.1: Der Klick-Zähler

Konzept: Klasse, `__init__`, `self`, Zustand

```
class Zaehler:
    def __init__(self):
        self.stand = 0

    def klick(self):
        self.stand += 1
        return self.stand

z = Zaehler()
z.klick()
z.klick()
print(z.klick())
```

**Frage:** Was gibt `print(z.klick())` aus?

**Hinweis:** Versteht man, dass das Objekt Zustand hat, der sich über Aufrufe hinweg verändert? Drei Aufrufe von `klick()` erhöhen `self.stand` auf 3.

#### Challenge 4.2: Zwei Sparschweine

Konzept: Mehrere Instanzen, Unabhängigkeit

```
class Spardose:
    def __init__(self):
        self.inhalt = 0

    def einwerfen(self, betrag):
        self.inhalt += betrag

a = Spardose()
b = Spardose()
a.einwerfen(50)
a.einwerfen(30)
b.einwerfen(10)
print(a.inhalt, b.inhalt)
```

**Frage:** Was wird ausgegeben?

**Hinweis:** Sind Instanzen wirklich unabhängig voneinander? Ja! `a.inhalt` ist 80, `b.inhalt` ist 10. Im Gegensatz dazu zeigt Challenge 4.3, was passiert, wenn man Klassenvariablen verwendet...

### Challenge 4.3: Der geteilte Trickhund

Konzept: Klassen- vs. Instanzvariablen (Falle!)

```
class Hund:
    tricks = []

    def __init__(self, name):
        self.name = name

    def lerne(self, trick):
        self.tricks.append(trick)

rex = Hund("Rex")
bello = Hund("Bello")
rex.lerne("Sitz")
print(bello.tricks)
```

**Frage:** Was gibt `print(bello.tricks)` aus?

**Hinweis:** Die OOP-Falle! `tricks` ist eine Klassenvariable – alle Instanzen teilen sich dieselbe Liste. Ergebnis: `["Sitz"]`. Direkte Brücke zu Challenge 1.2 (Doppelgänger-Liste): gleiche Mechanik, neuer Kontext! Fix: `tricks` sollte in `__init__` als `self.tricks = []` definiert werden.

### Challenge 4.4: Wer hat hier gezählt?

Konzept: Klassenvariable sinnvoll eingesetzt

```
class Teilnehmer:
    anzahl = 0

    def __init__(self, name):
        self.name = name
        Teilnehmer.anzahl += 1

anna = Teilnehmer("Anna")
ben = Teilnehmer("Ben")
clara = Teilnehmer("Clara")
print(Teilnehmer.anzahl)
```

**Frage:** Was wird ausgegeben?

**Hinweis:** Nach der Falle aus Challenge 4.3 jetzt die Pointe: Klassenvariablen sind nicht schlecht – man muss sie nur bewusst einsetzen. Hier zählt `anzahl` die Gesamtzahl aller erzeugten Instanzen. Ergebnis: 3.

## Challenge 4.5: Das Tierstimmen-Orchester

Konzept: Vererbung und Polymorphie

```
class Tier:
    def __init__(self):
        self.lebendig = True

    def spricht(self):
        return "..."

    def ist_lebendig(self):
        return self.lebendig

class Katze(Tier):
    def __init__(self):
        super().__init__()

    def spricht(self):
        return "Miau"

class Hund(Tier):
    def __init__(self):
        super().__init__()

    def spricht(self):
        return "Wuff"

tiere = [Katze(), Hund(), Tier(), Katze()]
print([t.spricht() for t in tiere])
```

**Frage:** Was wird ausgegeben?

**Hinweis:** Polymorphie zum Anfassen: gleicher Methodenname, verschiedenes Verhalten je nach Typ. Ergebnis: ["Miau", "Wuff", "...", "Miau"]. Hinweis: Die Methode `ist_lebendig(self)` nutzt den `self`-Parameter korrekt, um auf das Instanzattribut zuzugreifen.

## Challenge 4.6: Die Formenfamilie

Konzept: Vererbung mit eigenen Attributen, `super()`

```
class Form:
    def __init__(self, farbe):
        self.farbe = farbe

class Kreis(Form):
    def __init__(self, farbe, radius):
        super().__init__(farbe)
        self.radius = radius

    def flaeche(self):
        return 3.14159 * self.radius ** 2

k = Kreis("rot", 5)
print(k.farbe, int(k.flaeche()))
```

**Frage:** Was wird ausgegeben?

**Hinweis:** Farbe kommt von Form (Elternklasse), Fläche ist Kreis-spezifisch. Ergebnis: "rot 78".



## Teil 5: Code-Interpretation – OOP-Bausteine

---

Diese Aufgaben vertiefen das Verständnis grundlegender OOP-Muster: Zustandsmanagement, Objektinteraktion und Dunder-Methoden.

### 5.1 Die Batterie – Konstruktor und Zustand

Ein Objekt merkt sich Werte über die Lebensdauer der Variablen hinweg.

```
class MysteryA:
    def __init__(self, startwert):
        self.ladung = startwert

    def benutzen(self):
        if self.ladung > 0:
            self.ladung -= 1
            return True
        return False
```

**Was es macht:** Es simuliert eine Batterie, die sich bei jeder Benutzung entlädt, bis sie leer ist.

**Lernziel:** Die `__init__`-Methode (Konstruktor), `self` und wie Attribute einen Zustand speichern, der sich bei jedem Methodenaufruf verändert.

### 5.2 Der Rucksack – Objekte und Listen

Hier wird Wissen über Listen mit OOP-Konzepten kombiniert.

```
class MysteryB:
    def __init__(self):
        self.dinge = []

    def sammeln(self, x):
        if x not in self.dinge:
            self.dinge.append(x)

    def anzahl(self):
        return len(self.dinge)
```

**Was es macht:** Ein Inventar, das Gegenstände aufnimmt, aber Duplikate ignoriert – eine Art manuelles Set.

**Lernziel:** Attribute können auch komplexe Datenstrukturen wie Listen sein. Methoden verwalten diese internen Strukturen.

## 5.3 Der Vampir – Interaktion zwischen Objekten

Ein häufiger Aha-Moment: Man kann Objekte an Methoden anderer Objekte übergeben.

```
class MysteryC:
    def __init__(self, w):
        self.kraft = w

    def treffen(self, anderer):
        self.kraft += anderer.kraft
        anderer.kraft = 0
```

**Was es macht:** Ein Objekt "saugt" die Kraft eines anderen komplett auf – wie ein Vampir, Pac-Man oder eine Banküberweisung.

**Lernziel:** Die Interaktion zwischen Objekten: anderer hat denselben Aufbau wie self, und man kann auf dessen Attribute von außen zugreifen und sie verändern.

## 5.4 Das Echo – Dunder-Methoden

Dieses Snippet zeigt ein häufig genutztes Python-Feature, um Objekte lesbar zu machen.

```
class MysteryD:
    def __init__(self, text):
        self.t = text

    def __str__(self):
        return (self.t + " ") * 3
```

**Was es macht:** Bei print() wird der Text dreimal hintereinander ausgegeben – wie ein Echo.

**Lernziel:** Python ruft im Hintergrund "Dunder-Methoden" (Double Underscore) auf. `__str__` wird automatisch verwendet, wenn ein Objekt in einen String umgewandelt oder mit print() ausgegeben wird.

## Teil 6: OOP II – Beziehungen zwischen Klassen

Hier geht es um die vier Arten von Beziehungen zwischen Klassen: Vererbung (is-a), Assoziation, Aggregation und Komposition (has-a). Jedes Rätsel bricht eine dieser Beziehungen auf wenige Zeilen herunter.

### 6.1 Das Upgrade – Vererbung (is-a)

Eine Klasse übernimmt das Verhalten einer anderen und erweitert es, ohne den alten Code neu schreiben zu müssen.

```
class Basis:
    def rechne(self, x):
        return x * 2

    def hallo(self):
        return "Hi"

class MysteryE(Basis):
    def rechne(self, x):
        altes_ergebnis = super().rechne(x)
        return altes_ergebnis + 10
```

**Was es macht:** `MysteryE.rechne(5)` ergibt 20 ( $5 \cdot 2 + 10$ ). `MysteryE.hallo()` funktioniert, obwohl die Methode in `MysteryE` gar nicht definiert ist.

**Lernziel:** Vererbung (Klammer bei der Klassendefinition), geerbte Methoden und `super()` zum Aufrufen der Elternmethode.

### 6.2 Der flüchtige Kontakt – Assoziation

Assoziation ist die lockerste Beziehungsform: Ein Objekt nutzt ein anderes, aber keines "besitzt" das andere.

```
class MysteryF:
    def __init__(self, name):
        self.n = name

    def begruessen(self, anderer):
        print(self.n + " winkt " + anderer.n)
```

**Was es macht:** Ein Objekt nutzt kurzzeitig die Eigenschaften eines anderen Objekts (z. B. `objekt1.begruessen(objekt2)`).

**Lernziel:** Objekte können als Parameter in Methoden übergeben werden. Sie existieren völlig unabhängig und kennen sich nur für den Moment des Methodenaufrufs.

## 6.3 Der Club – Aggregation (has-a, lose)

Bei der Aggregation besitzt das Ganze seine Teile, aber die Teile können auch ohne das Ganze existieren.

```
class MysteryG:
    def __init__(self):
        self.mitglieder = []

    def aufnehmen(self, person):
        self.mitglieder.append(person)

    def auflösen(self):
        self.mitglieder.clear()
```

**Was es macht:** Es verwaltet eine Liste von Objekten (z. B. Spieler in einem Team). Wenn das Team aufgelöst wird, existieren die Personen-Objekte weiterhin.

**Lernziel:** Objekte werden von außen hineingereicht. Das ist der Unterschied zur Komposition.

## 6.4 Der Motorblock – Komposition (has-a, streng)

Komposition ist die strenge Form: Das Ganze erstellt seine Teile selbst. Wird das äußere Objekt gelöscht, sterben auch die inneren.

```
class Herz:
    def schlagen(self):
        return "Bumm Bumm"

class MysteryH:
    def __init__(self):
        self.antrieb = Herz()

    def leben(self):
        return self.antrieb.schlagen()
```

**Was es macht:** Sobald ein MysteryH-Objekt erzeugt wird, erstellt es automatisch sein eigenes Herz-Objekt.

**Lernziel:** Objekte können innerhalb des Konstruktors einer anderen Klasse instanziiert werden. Wird das äußere Objekt gelöscht, wird auch das innere vom Garbage Collector entfernt, da keine externe Referenz existiert.

## Anhang: Klassische Algorithmen im PyTorch-Kontext

---

Die folgenden Algorithmen sind in einem typischen PyTorch-Trainingsscript (z. B. mit Early Stopping) versteckt. Sie zeigen, dass die in diesem Skript gelernten Grundlagen direkt in der Praxis der KI-Entwicklung zum Einsatz kommen.

### 1. Lineare Minimumsuche (Laufendes Minimum)

Wird in der Early-Stopping-Logik verwendet. Der Algorithmus prüft fortlaufend, ob der aktuelle Validation Loss kleiner ist als der bisherige Bestwert.

### 2. Lineare Maximumsuche (Argmax)

Wird bei `torch.max(outputs, 1)` zur Klassifizierung aufgerufen. Der Algorithmus sucht den höchsten Wert und gibt dessen Index (die vorhergesagte Klasse) zurück.

### 3. Deterministischer Endlicher Automat (DFA)

Die Early-Stopping-Struktur schaltet je nach Zustand der Loss-Entwicklung (Verbesserung → Zähler-Reset, Verschlechterung → Zähler hochzählen) hin und her, bis der Abbruchzustand erreicht ist.

### 4. Sortialgorithmen (Timsort)

Der LabelEncoder sammelt die Text-Kategorien und sortiert sie intern alphabetisch, um ihnen feste Zahlenindizes zuzuweisen.

### 5. Fisher-Yates-Shuffle

Wird durch `shuffle=True` im DataLoader ausgeführt. Er mischt die Trainingsdaten vor jeder Epoche mathematisch unverzerrt durch.

### 6. Graph-Traversierung (Tiefensuche)

Arbeitet bei `loss.backward()`. PyTorch durchläuft den dynamisch aufgebauten Operationsgraphen rückwärts, um Gradienten zu berechnen.

### 7. Dynamische Programmierung

Der Backpropagation-Algorithmus speichert berechnete Zwischengradienten und verwendet sie für vorhergehende Schichten wieder, anstatt alles redundant neu zu berechnen.

### 8. Partitionierung (Daten-Splitting)

`train_test_split` partitioniert die Gesamtdatenmenge nach festen Prozentwerten in separate, gleichverteilte Mengen (Train, Validation, Test).